

Sistema de Sorteio e Distribuição de Processos Judiciais

Documentação técnica e didática do projeto

Este documento explica, parte por parte, como o sistema funciona, qual a função de cada arquivo, qual a lógica por trás das decisões e como demonstrar cada um dos perfis de usuário.

Stack utilizada

Back-end: Python 3.11+, FastAPI, SQLAlchemy 2, SQLite, Pydantic v2, JWT (python-jose + passlib), APScheduler, pytest.

Front-end: HTML5, JavaScript puro (sem framework), Bootstrap 5 via CDN, Bootstrap Icons, fetch API, localStorage.

1. Visão geral do sistema

O sistema simula a rotina de um tribunal pequeno: um **assistente** cadastra processos, o sistema **sorteia** esses processos entre 8 **juízes**, e o **assessor** de cada juiz pode editar os processos sob autorização. O perfil **TI** administra o sistema todo, e o **juiz externo** é criado sob demanda quando os 8 juízes internos têm conflito com o mesmo processo.

1.1 Perfis de usuário e o que cada um faz

Perfil	Função no sistema
TI	Administra tudo: usuários, sorteio manual, designação de juiz externo, listagem geral.
JUIZ	Vê seus processos, registra conflito de interesse, autoriza o assessor a editar, marca reabertura.
ASSESSOR	Vê os processos do juiz ao qual está vinculado e edita quando o juiz autoriza.
ASSISTENTE	Cadastra pessoas (PF/PJ) e processos. Não enxerga juiz nem status real.
JUIZ_EXTERNO	Vê só o processo específico que o TI designou para ele.

1.2 O que é um processo aqui

Um **processo** tem um número único, descrição, uma **pessoa** ligada (autor ou réu, PF ou PJ), e um **nível** de complexidade:

- **Nível 1 (Básico)**: pode se repetir no mesmo juiz à vontade.
- **Nível 2 (Intermediário)**: pode se repetir também.
- **Nível 3 (Complexo)**: tem **rodízio**. Os 8 juízes recebem um cada antes que algum receba o segundo, e ninguém pega dois complexos em sequência.

1.3 Status que um processo pode ter

Status	Significado
PENDENTE	Cadastrado mas ainda não sorteado.
SORTEADO	Já tem juiz responsável.
AGUARDANDO_JUIZ_EXTERNO	Todos os 8 juízes têm conflito; o TI precisa designar um juiz externo.
ENCERRADO	Reservado para uso futuro.

2. Estrutura de pastas

A organização do projeto é simples e plana — cada arquivo tem um papel claro:

```
sistema_sorteio/
■■■ main.py                # liga tudo: FastAPI, rotas, frontend, scheduler
■■■ database.py            # conexão com o SQLite
■■■ models.py              # tabelas do banco
■■■ schemas.py             # validação dos dados de entrada/saída (Pydantic)
■■■ auth.py                # senha, JWT, controle de perfil
■■■ sorteio.py             # o algoritmo do sorteio
■■■ scheduler.py           # roda o sorteio todo dia útil às 10h
■■■ seed.py                # popula o banco com usuários e exemplos
■■■ routers/               # endpoints separados por assunto
■   ■■■ usuarios.py
■   ■■■ pessoas.py
■   ■■■ processos.py
■   ■■■ sorteio_router.py
■   ■■■ conflitos.py
■■■ frontend/
■   ■■■ login.html        # tela de login
■   ■■■ app.html          # aplicação principal (tudo numa página só)
■   ■■■ css/style.css     # ajustes visuais mínimos
■   ■■■ js/
■       ■■■ api.js        # função wrapper que chama a API com o token
■       ■■■ app.js        # lógica de telas, formulários, listas
■■■ tests/                # 12 testes automatizados
■■■ requirements.txt
■■■ README.md
```

Por que essa estrutura? Cada pasta agrupa um tipo de coisa. Quem mexe na regra do sorteio abre *sorteio.py*. Quem quer mudar a tela de login, *frontend/login.html*. Não há mistura.

3. O banco de dados

Usamos **SQLite** — um banco em arquivo único (*sorteio.db*). Não precisa instalar servidor, não precisa configurar. Ideal para o trabalho acadêmico.

3.1 database.py — a conexão

Esse arquivo só faz três coisas:

- Cria a **engine** apontando para o arquivo *sorteio.db*.
- Cria a **SessionLocal**, que é como uma "sessão" de conversa com o banco.
- Define a função **get_db()** que abre e fecha uma sessão automaticamente em cada requisição.

```
engine = create_engine("sqlite:///./sorteio.db", ...)
SessionLocal = sessionmaker(autocommit=False, autoflush=False, bind=engine)

def get_db():
    db = SessionLocal()
    try:
        yield db
    finally:
        db.close()
```

O *yield* em vez de *return* é o que permite o FastAPI "injetar" essa sessão dentro de cada endpoint, e ainda assim fechar a conexão depois — mesmo se der erro.

3.2 models.py — as tabelas

Cada classe vira uma tabela. Resumo:

Tabela	O que guarda
usuarios	Login, senha (hash), nome, perfil, ativo.
juizes	Liga um usuário ao papel de juiz; tem tipo PRINCIPAL ou EXTERNO.
assessores	Liga um usuário a um juiz específico.
peessoas	PF (CPF) ou PJ (CNPJ) que aparece nos processos.
processos	O processo em si: número, descrição, nível, pessoa, status, juiz.
conflitos_interesse	Quando um juiz declara que não pode julgar um processo.
prioridades_compensacao	Marca que um juiz "ficou devendo" um processo depois de declarar conflito.
acesso_juiz_externo	Quais processos cada juiz externo pode ver.
log_sorteio	Histórico de todo sorteio: data, processo, juiz, motivo.
autorizacoes_edicao	Tokens que o juiz dá ao assessor para editar um processo (uso único).

Por que tabelas separadas para juízes e assessores se eles já têm uma linha em *usuarios*?
Para guardar informações que só fazem sentido para esse papel — por exemplo, qual juiz um assessor atende, ou se um juiz é principal ou externo. É a separação entre *quem você é* (usuário) e *qual seu papel* (juiz/assessor).

4. Autenticação (auth.py)

Quando o usuário faz login, devolvemos um **token JWT**. Esse token é uma string que o front-end guarda no *localStorage* e envia em todo pedido seguinte no cabeçalho *Authorization*.

4.1 Senha

Nunca guardamos a senha em texto. Quando o usuário é criado, passamos a senha pelo **bcrypt** (via *passlib*), que devolve um hash. No login, comparamos a senha digitada com o hash.

```
def hash_senha(s): return pwd_ctx.hash(s)
def verificar_senha(s, h): return pwd_ctx.verify(s, h)
```

4.2 Token JWT

O token leva dentro dele o ID do usuário e uma data de expiração (12 horas). Tudo isso é assinado com uma chave secreta, então o servidor consegue confirmar que o token veio dele mesmo e não foi forjado.

```
def criar_token(usuario_id):
    payload = {"sub": str(usuario_id),
               "exp": datetime.utcnow() + timedelta(minutes=720)}
    return jwt.encode(payload, SECRET_KEY, algorithm="HS256")
```

4.3 Controle de perfil

Cada endpoint usa uma **dependência** que pega o token, descobre o usuário e checa o perfil. Se o perfil não bater com o que aquele endpoint exige, devolve erro **403**.

```
def exige_perfil(*perfis):
    def dep(u = Depends(usuario_atual)):
        if u.perfil not in perfis:
            raise HTTPException(403, "Acesso negado")
        return u
    return dep

# uso:
@router.post("/usuarios/")
def criar(..., _ = Depends(exige_perfil(PerfilEnum.TI))):
    ...
```

Por que usar JWT em vez de sessão tradicional? Como o front-end é estático (HTML+JS), não temos um "estado de sessão" no servidor. O JWT é auto-suficiente: o cliente o guarda, manda em cada request, e o servidor valida sem precisar consultar nada.

5. O coração do sistema: o algoritmo de sorteio

O arquivo `sorteio.py` tem a função `executar_sorteio(db)`. Ela percorre todos os processos que estão no status PENDENTE, em ordem de cadastro, e tenta atribuir cada um a um juiz seguindo cinco etapas em cascata.

5.1 As cinco etapas, em linguagem simples

Etapas 1 — Reabertura: se o processo foi reaberto e tem um juiz anterior ativo, ele volta direto pra esse juiz. Não cai no sorteio. A ideia é manter o mesmo juiz para garantir continuidade.

Etapas 2 — Conflito: tiramos da lista qualquer juiz que tenha declarado conflito de interesse naquele processo. Se sobrar zero juiz, o processo vai para AGUARDANDO_JUIZ_EXTERNO e o TI designa alguém de fora.

Etapas 3 — Rodízio dos complexos (só nível 3): para evitar sobrecarregar um juiz, processos complexos seguem duas regras: (a) não vai pro último juiz que pegou um complexo, e (b) precisa ser alguém que ainda não pegou complexo nessa "rodada". Quando os 8 juízes pegaram um, a rodada zera.

Etapas 4 — Compensação: se algum juiz declarou conflito num processo desse mesmo nível antes, ele tem prioridade agora — recebe o processo automaticamente. Isso compensa a injustiça de ele ter perdido um caso anterior.

Etapas 5 — Sorteio aleatório: se nenhuma das regras acima decidiu, escolhemos um juiz com `random.choice()` entre os elegíveis.

5.2 Por que tudo numa transação só?

Se uma das etapas falhar no meio do processo, queremos **desfazer tudo** — não pode ficar metade dos processos sorteados e a outra metade não. Por isso colocamos toda a função num `try/except`: se der erro, `db.rollback()` desfaz; se chegar até o fim, `db.commit()` grava.

5.3 Por que precisamos do `db.flush()` dentro do loop?

O SQLAlchemy, por padrão nas nossas configurações, não envia os `INSERTs` para o banco antes do commit. Como dentro do loop perguntamos "qual foi o último juiz que pegou um complexo?" — precisamos que o log da iteração anterior já esteja visível na consulta. O `flush()` empurra os dados pendentes para o banco sem fechar a transação.

Esse foi exatamente o bug que apareceu nos testes na primeira execução: o teste do rodízio achava só 5 juízes distintos em vez de 8. A correção foi colocar `db.flush()` depois de adicionar cada log.

5.4 Cada sorteio gera um log

Toda atribuição de juiz vira uma linha em *log_sorteio*. Isso serve de auditoria — qualquer pessoa consegue conferir depois o motivo de cada distribuição.

Motivo	Quando acontece
SORTEIO_NORMAL	Sorteio aleatório (caso comum).
REABERTURA	Processo voltou pro juiz anterior.
PRIORIDADE_COMPENSACAO	Juiz tinha ficado devendo um processo desse nível.
DESIGNACAO_EXTERNA	TI designou um juiz de fora manualmente.

5.5 Quando o sorteio roda automaticamente

O arquivo **scheduler.py** usa a biblioteca **APScheduler** para agendar o sorteio às **10:00 da manhã**, no fuso de Belém, **todo dia útil**. A biblioteca *holidays* nos diz se a data é feriado nacional. Em sábado, domingo ou feriado o job simplesmente não faz nada.

```
sched.add_job(_job, CronTrigger(hour=10, minute=0,
                                timezone="America/Belem"))
```

O TI também pode disparar o sorteio na hora pelo botão "Executar sorteio agora", que chama o endpoint *POST /sorteio/executar*.

6. Os endpoints da API

Cada router cuida de uma área. O FastAPI gera automaticamente o **Idocs** (Swagger) onde você pode testar cada endpoint sem precisar do front-end.

6.1 /auth/login

Recebe *login* e *senha*. Se baterem, devolve um JSON com *token*, perfil, nome, e os IDs auxiliares (*juiz_id* ou *assessor_id* quando aplicável).

6.2 /usuarios/* — só TI

CRUD de usuários. O TI cria juízes, assessores, assistentes. Quando cria um juiz, automaticamente também cria o registro em *juizes*.

6.3 /pessoas/*

Assistente e TI cadastram pessoas (PF/PJ). Os outros perfis só listam. A validação básica do tamanho do CPF (11 dígitos) e do CNPJ (14 dígitos) acontece tanto no back quanto no front.

6.4 /processos/*

Aqui mora a maior parte da lógica de permissão. Resumo:

Ação	Quem pode
Criar processo	Só ASSISTENTE.
Editar (PENDENTE)	Só o ASSISTENTE que criou.
Editar (SORTEADO)	Só o ASSESSOR do juiz, com autorização.
Listar	Cada perfil vê uma fatia diferente.
Marcar reabertura	TI ou JUIZ.
Autorizar edição	Só o JUIZ responsável.

Na hora de listar, fizemos uma função *serializar* que monta o JSON. Para o assistente, esse JSON é diferente: escondemos *juiz_id*, *sorteado_em* e mostramos um status genérico ("cadastrado" ou "em processamento"). Isso garante que ele não fica sabendo qual juiz pegou o processo dele.

6.5 /conflitos/*

Quando um **juiz** envia um conflito, três coisas acontecem ao mesmo tempo: (1) registra o conflito com o arquivo anexado; (2) cria uma prioridade de compensação para ele; (3) o processo volta para PENDENTE. No próximo sorteio, aquele juiz não aparece como elegível para esse processo, mas tem prioridade no próximo processo do mesmo nível.

Quando o **TI** designa um juiz externo, criamos no mesmo movimento: o usuário (perfil JUIZ_EXTERNO), o juiz (tipo EXTERNO), o registro em *acesso_juiz_externo* e o log do sorteio com motivo DESIGNACAO_EXTERNA.

6.6 /sorteio/*

Dois endpoints: *POST /sorteio/executar* roda o algoritmo (só TI); *GET /sorteio/logs* lista o histórico (TI ou JUIZ).

7. O front-end

Tudo em **HTML + JavaScript puro**, sem build, sem framework. O FastAPI serve os arquivos estáticos da pasta *frontend/*.

7.1 Duas páginas só

- **login.html**: formulário pequeno; ao enviar, chama *POST /auth/login*, guarda o retorno no *localStorage* e redireciona para */app*.
- **app.html**: uma única página com várias *<section>*, cada uma é uma "tela". O JavaScript esconde todas e mostra só a que está ativa. É uma SPA caseira.

7.2 api.js — o wrapper que fala com o servidor

Essa função é a única que faz *fetch()* no projeto. Ela sempre coloca o token no cabeçalho, sabe quando os dados são JSON e quando são FormData (para upload de arquivo), e se receber 401 já desloga e manda pra tela de login.

```
async function api(path, opts = {}) {
  const auth = getAuth();
  const headers = opts.headers || {};
  if (auth?.token) headers['Authorization'] = `Bearer ${auth.token}`;
  // se for um objeto JS comum, converte pra JSON
  if (!(opts.body instanceof FormData) && opts.body
    && typeof opts.body !== 'string') {
    headers['Content-Type'] = 'application/json';
    opts.body = JSON.stringify(opts.body);
  }
  const r = await fetch(API_BASE + path, { ...opts, headers });
  if (r.status === 401) {
    clearAuth();
    window.location.href = '/';
    return;
  }
  ...
}
```

7.3 app.js — a lógica das telas

Esse arquivo tem três partes principais:

- Um **dicionário MENUS** que diz quais opções cada perfil vê na sidebar. Assim, o menu de "Sorteio" só aparece para TI, e "Cadastrar Processo" só para o assistente.
- A função **mostrarTela(id)**, que esconde todas as *<section>* e mostra só uma. Depois chama **carregarTela(id)** que busca os dados da API para popular a tabela ou form.
- Os **handlers** dos formulários (login do usuário, cadastro de pessoa, cadastro de processo, executar sorteio, registrar conflito, etc.), que chamam *api()* e mostram um alerta de sucesso ou erro em cima da tela.

Por que não usar React/Vue? O escopo é pequeno e o objetivo do trabalho é mostrar as regras do sorteio, não a arquitetura do front. JS puro funciona muito bem aqui e tira a complexidade de build, dependências, etc.

8. Dados de demonstração (seed.py)

Para não precisar criar cada usuário e processo na mão, o **seed.py** zera o banco e popula com:

Quantidade	O quê	Login / senha
1	TI	ti / ti123
8	Juizes principais	juiz1..juiz8 / juiz123
8	Assessores (um por juiz)	assessor1..assessor8 / assessor123
2	Assistentes	assistente1..2 / assistente123
5	Pessoas (3 PF + 2 PJ)	—
10	Processos PENDENTES misturando níveis 1, 2 e 3	—

Rodar com: *python seed.py*

9. Os testes automatizados

São **12 testes**, organizados em dois arquivos. Todos passam. Para rodá-los: *pytest*

9.1 tests/test_sorteio.py — 7 testes da regra

- **Reabertura**: processo reaberto volta para o juiz anterior, sem cair no sorteio.
- **Não-consecutividade**: dois complexos seguidos não vão para o mesmo juiz.
- **Rodízio completo**: 8 processos complexos → exatamente 1 para cada juiz.
- **Conflito tira o juiz**: se 7 dos 8 juízes têm conflito, o oitavo recebe o processo.
- **Compensação**: o juiz que tinha prioridade recebe o próximo do mesmo nível, e a prioridade fica marcada como consumida.
- **Conflito de todos**: o processo vai para AGUARDANDO_JUIZ_EXTERNO.
- **Níveis 1 e 2 podem repetir**: com 20 processos básicos para 8 juízes, é normal que algum receba mais de um.

9.2 tests/test_permissoes.py — 5 testes de acesso

- Assistente não vê o status real, nem o *juiz_id*.
- Assessor só vê processos do seu juiz.
- Assessor não consegue editar sem autorização (403).
- Juiz externo só enxerga o processo que lhe foi designado.
- Juiz comum não consegue criar usuário (403).

Os testes usam SQLite em memória (não tocam o *sorteio.db* real) e o *TestClient* do FastAPI, que simula requisições HTTP sem subir servidor.

10. Como rodar e demonstrar

10.1 Subir o sistema

```
pip install -r requirements.txt
python seed.py
uvicorn main:app --reload
```

Abrir **<http://localhost:8000/>**. A documentação interativa da API fica em **<http://localhost:8000/docs>**.

10.2 Roteiro de demonstração

1. Como TI (*ti / ti123*)

Login → Dashboard mostra os totais → vai em "Sorteio" → clica "Executar sorteio agora". Os 10 processos PENDENTES são distribuídos e aparecem na tabela de logs.

2. Como Juiz (*juiz1 / juiz123*)

Login → "Meus Processos": vê os que ganhou no sorteio. Pode usar "Marcar reabertura" (e ao executar o sorteio de novo como TI, o processo volta para juiz1).

Em "Registrar Conflito": escolhe um processo, escreve a justificativa, anexa qualquer arquivo. O processo volta para PENDENTE e uma prioridade é registrada.

3. Como Assessor (*assessor1 / assessor123*)

Login → "Processos do Meu Juiz": só vê os do juiz1. Se clicar em "Editar" sem autorização → erro 403.

Voltar como juiz1 → "Autorizar edição" naquele processo → voltar como assessor1 e editar. Funciona uma vez só.

4. Como Assistente (*assistente1 / assistente123*)

Login → "Cadastrar Pessoa", depois "Cadastrar Processo". Em "Meus Cadastros" aparece só o que ele cadastrou, sem mostrar juiz e com status genérico ("cadastrado" ou "em processamento").

5. Cenário do juiz externo

Para forçar: como cada juiz, registrar conflito no mesmo processo (precisa estar com aquele juiz naquele momento). Quando os 8 conflitos forem registrados e o sorteio rodar, o processo cai em AGUARDANDO_JUIZ_EXTERNO. O TI vai em "Processos", filtra por esse status e clica em "Designar externo". Pronto — o novo login do juiz externo entra no sistema e vê só esse processo.

11. Pontos importantes para a apresentação

- **Separação por perfil:** a regra de quem pode o quê está espalhada no back-end (dependências do FastAPI), no banco (qual usuário é juiz/assessor) e no front-end (sidebar montada conforme o perfil).
- **Auditabilidade:** cada sorteio gera um log permanente com data, juiz, processo e motivo. Nunca se perde a história de como um processo chegou a quem.
- **Justiça do sorteio:** a combinação das etapas (reabertura → conflito → rodízio → compensação → sorteio) garante que ninguém é sobrecarregado e que situações imprevistas (conflito, reabertura) são tratadas de forma previsível.
- **Atomicidade:** todo o sorteio é uma só transação. Se algo falhar no meio, nada é gravado. Isso protege a integridade dos dados.
- **Simplicidade do front-end:** tudo HTML/JS puro. Para mostrar uma tela, basta dar um *display: block* nela. Para falar com o servidor, uma única função *api()* faz tudo.
- **Testes garantem que as regras funcionam:** 12 testes cobrem o sorteio e as permissões. Rodar *pytest* antes da apresentação dá segurança de que nada quebrou.

Em uma frase: O sistema é um servidor FastAPI que expõe uma API REST protegida por JWT, com regras de sorteio implementadas em uma função pura testável, persistência em SQLite, e uma interface web simples em Bootstrap que mostra para cada perfil exatamente o que ele precisa ver.